

Dummy PDF file

JavaScript is a flexible multi-paradigm programming language largely used in the web-development space for both front-end and back-end applications. Whereas HTML and CSS describe the elements on a webpage, code written in JavaScript makes them interactive. A framework such as NodeJS allows back-end code to be written in JavaScript. The tutorials below provide tips and tricks on how to get started with Javascript.

An **Abstract Data Type** (ADT), as the name suggests, is an abstract understanding of a data structure. An ADT is defined through its behavior and characteristics, particularly in terms of what data can be stored into it, the operations that can be performed on this data, and the behavior of these operations. For example, stacks and queues can be internally implemented using linked-lists made up of nodes or [arrays](#). However, the primary function of a stack is to be a last in, first out (LIFO) data structure and the primary function of a queue is to be a first in, first out (FIFO) data structure. The behavior, from the point of the user, remains intact, regardless of the internal implementation either using linked-lists or arrays. If the user was interacting with a stack, the user will simply worry about pushing data onto the stack or popping data off the stack. The user won't need to have the knowledge of how that stack is working internally.

In contrast to the data structures, which are specific and detailed implementations that deal with how the data structure does its job, an ADT focuses on *what it does* and not how it does its job. In short, the ADT defines what that particular data construct must do and the data structure is the concrete implementation of that construct.

An analogy to explain ADTs in terms of web development would be [CRUD](#) (abbreviated as create, read, update and delete) APIs. The user of any CRUD API has to simply know what request method (GET, POST, PUT/PATCH, or DELETE) should they send, and if they followed the rules of the API, the API server would send data back. The user didn't have to worry about the internal workings of the API server. They simply had to know the rules of interactions and behavior of a CRUD API. In this case, the CRUD API is functioning as an ADT *from the perspective of the user*.

There are no specific rules which force the implementation of particular methods and operations in a particular ADT. This is decided based on the requirements in a use-case scenario and ultimately by design choice.

Why use ADTs?

There are 3 general advantages of using ADTs, listed as follows:

Encapsulation

An ADT will provide certain methods and properties. And the knowledge of these methods and properties is all the user will need to successfully operate with the ADT.

Compartmentalization

The code that is using the ADT will not have to be changed even if the internal workings of the ADT have been changed. The change in the ADT is isolated and compartmentalized.

Adaptability

Real world programs continue to evolve with ever changing requirements and new constraints. Differently implemented ADTs, with all the same properties and methods, can thus be used interchangeably. For example, consider a linked list created using arrays that contained names of the patients in a hospital. It's later decided to include all the information about the patient in the linked list. Then, a linked list implemented using class based nodes with all the necessary fields would serve as a much better replacement as compared to the linked list that is simply using arrays. Therefore, ADTs can adapt to the situation in which they're used.